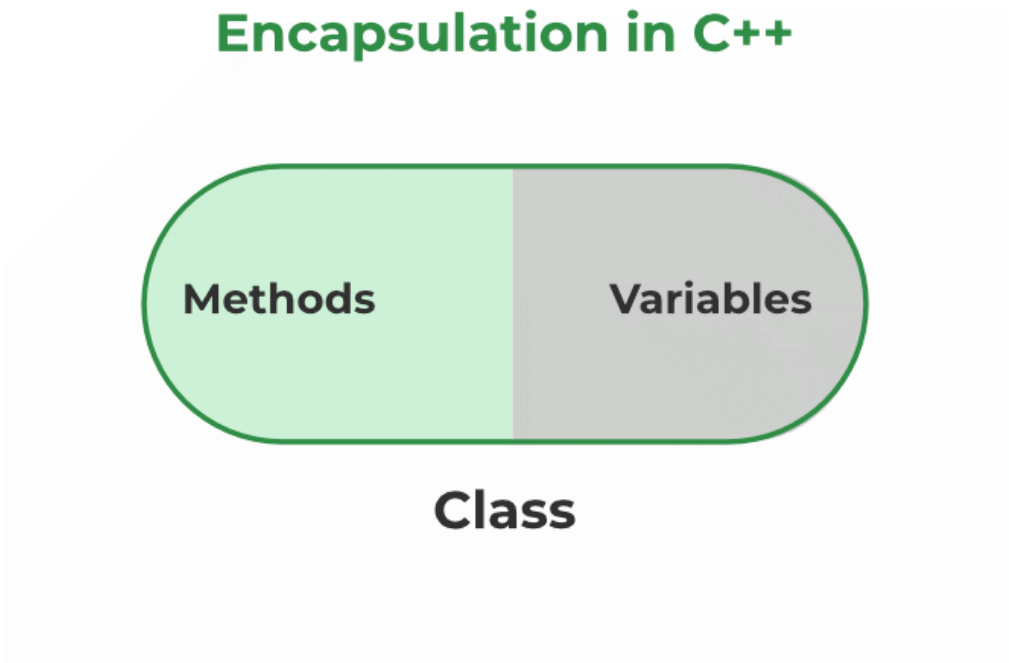
Object Oriented Programming(C++)
(CSE 1211)

Encapsulation

Md. Mahfuzur Rahman
Lecturer,
Prime University

Encapsulation in C++

- **Encapsulation** means combining data and the functions that work on that data into a single unit, like a class. In Object-Oriented Programming, it helps keep things organized and secure.



Encapsulation

Encapsulation is a process of

1. combining variables and functions in a single unit (class)
2. Protecting data by declaring them as private

Private data will be hidden from other classes and they can only be accessed through public function of their current class. That is known as data hiding.

```
class Student {
    private :
        int id;
        string name;
    public :
        void display ()
        {
         cout << id << endl;
         cout << name <<endl;
        }

};
```

```
int main()
{
    Student s1;
    s1.id = 101;
    s1.name = "Suparna";
    s1.display();
    getch();

}
```

Encapsulation

```
class Student {
        private :
          string name;
        public :
        void setName(string x)
        {
          name = x;
        }
        string getName ()
        {
          return name;
        }

};
```

```
int main()
{
        Student s1;
        s1.setName("Suparna");


}
```

Encapsulation

```cpp
#include<iostream>
using namespace std;

class Student
{
private:
    int id;
    string name;

public:
    void getData(int a, string b)
    {
        id = a;
        name = b;
    }
    void setData()
    {
        cout << id << " " << name << endl;
    }

};
```

```cpp
int main()
{
    Student ob;
    ob.getData(101, "Albatross");
    ob.setData();

}
```

Encapsulation

```cpp
#include <iostream>
using namespace std;

class Rectangle {
   private:

    int length;
    int breadth;

   public:

    void setLength(int len) {
      length = len;
    }

    void setBreadth(int brth) {
      breadth = brth;
    }

    int getLength() {
      return length;
    }

    int getBreadth() {
      return breadth;
    }
    int getArea() {
      return length * breadth;
    }
};
```

```cpp
int main() {

  Rectangle rectangle1;

  rectangle1.setLength(8);

  rectangle1.setBreadth(6);

  cout << "Length = " << rectangle1.getLength() << endl;

  cout << "Breadth = " << rectangle1.getBreadth() << endl;

  cout << "Area = " << rectangle1.getArea();

  return 0;
}
```

Encapsulation in C++

For example, think of a company with different departments—finance, sales, and accounts. Each department handles its own tasks and data. The finance department deals only with financial records, and the sales department handles only sales. Just like that, in programming, each class manages its own data and operations, keeping everything separate and protected. That's encapsulation.

In C++, OOPs encapsulation is implemented using classes and **access specifiers** that keeps the data, and the manipulating methods enclosed inside a single unit. The direct advantage of this packaging is that only the required components are visible to the user, and other information is hidden.

Encapsulation

The following are important points about encapsulation.

- **Better Code Management:** We can change data representation and implementation any time without changing the other codes using it if we keep method parameters and return values the same. With encapsulation, we ensure that no other code would have access to implementation details and data members.
- **Simpler Parameter Passing:** When we pass an object to a method, everything (associated data members and methods are passed along). We do not have to pass individual members.
- **getter and setter:** getter (display the data) and setter method (modify the data) are used to provide the functionality to access and modify the data, and the implementation of this method is hidden from the user. The user can use this method, but cannot access the data directly.

Advantages of Encapsulation:

Data Hiding: Internal object details are hidden from the outside world using access modifiers (like private), preventing unauthorized access.

Improved Maintainability: Changes to internal code can be made with minimal impact on external code that uses the class, as long as the interface (public methods) remains the same.

Reusability: Encapsulated code is modular and self-contained, making it easy to reuse in other parts of a program or in different projects.

Improved Code Organization: Related variables and functions are grouped logically inside the same class, improving readability and structure.

Flexibility and Control: By exposing only specific methods, the developer controls how data is accessed or modified. You can add validation or logic inside setters to ensure proper usage.

Benefits of Encapsulation

- **Data Security:** Protects data from external misuse.
- **Flexibility and Maintainability:** Allows internal implementation details to change without affecting external code that uses the class.
- **Code Reusability:** Encapsulated classes can be easily reused in different parts of a program or in other projects.
- **Reduced Complexity:** Simplifies the interface for users of the class, as they only need to interact with the public methods.

Advantages of Encapsulation

- **Better Testing and Debugging:** Encapsulated classes are easier to test individually (unit testing), and bugs are easier to isolate.

- **Data Integrity**: With the help of setter and getter methods we can ensure that the correct data is assigned to the variable.

- **Testing code is easy:** Encapsulated code is easy to test for unit testing.

- **Freedom of the details:** one of the major advantages of encapsulation is that it gives the programmer freedom to implement the details of a system. The only constraint on the programmer is to maintain the abstract interface that outsiders see.

Disadvantages of Encapsulation

- **Increased Code Complexity:** Writing getters and setters for every variable might feel repetitive and increase code size, especially in simple programs.

- **Performance Overhead:** Using methods instead of direct variable access can introduce slight overhead in performance-critical applications.

- **Overuse of Accessors Can Break Encapsulation:** If all fields have public getters and setters without restrictions, then the benefits of encapsulation are lost—it becomes just data exposure in disguise.